Advanced Prompt Engineering - III

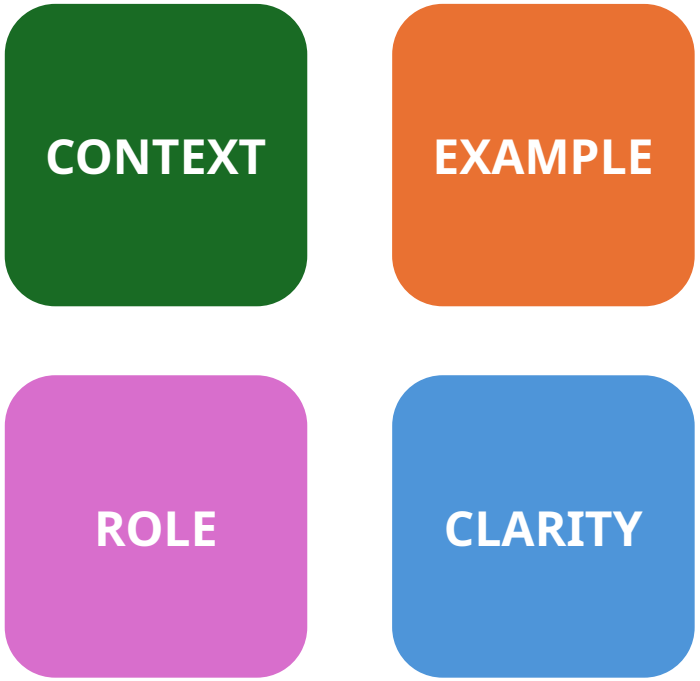
Session 3 – January 28th, 2025

3

- Quick Recap
- Live Demo of Popular Techniques
- Break
- Probing Techniques
- Live Demo using Probing Techniques
- Break
- Co-pilot Hands-on
- Q&A
- Empathy & Ethical Practices
- AI Error Handling & Closing Notes
- Q&A

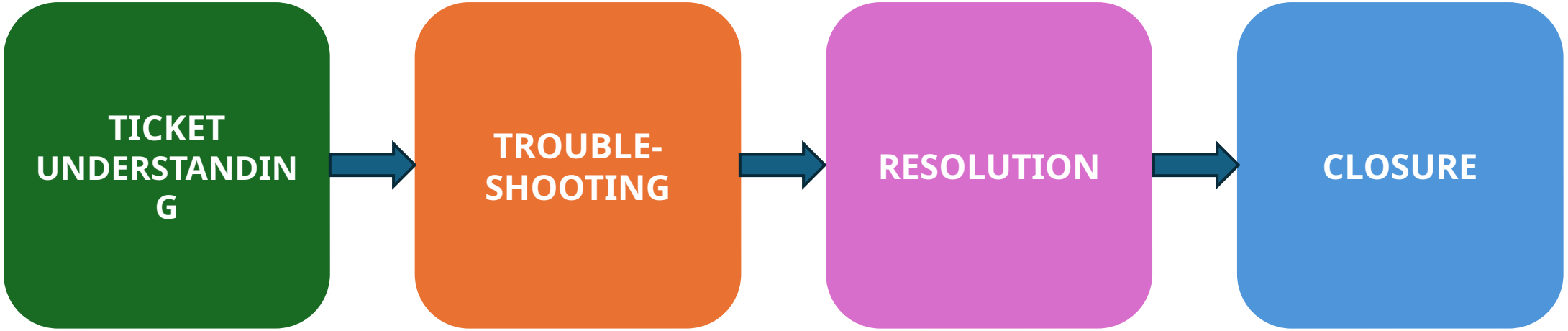
QUICK RECAP

Quick Recap



1. **<u>Context:</u>** Always provide a background story for the problem at hand.
2. **<u>Examples:</u>** Provide examples - help model mimic
3. **<u>Assign Role:</u>** Provide a role to the model to guide and tailor its response.
4. **<u>Clarity:</u>** Clearly define what you want the Copilot to do.

Quick Recap


TICKET UNDERSTANDIN G
TROUBLE-SHOOTING
RESOLUTION
CLOSURE

Quick Recap

Text Extraction:
Identify key details from customer tickets.

Text Summarization:
Summarize long, complex tickets for quick understanding.

Text Generation:
Generate clear, empathetic responses or troubleshooting steps.

Knowledge Retrieval:
Retrieve relevant solutions from internal knowledge bases.

Anomaly Detection:
Identify unusual patterns or issues in tickets.

Multilingual Support:
Break language barriers with real-time translations.

Sentiment Analysis:
Understand customer tone and urgency.

Quick Recap

Stage	Capability	Outcomes
Ticket Understanding	Text Extraction, Sentiment Analysis	Extract version details; Identify negative sentiment.
Troubleshooting	Knowledge Retrieval, Anomaly Detection	Identify root causes from logs or patterns.
Resolution	Text Generation, Multilingual Support	Provide step-by-step guidance; Translate responses.
Closure	Sentiment Analysis, Text Summarization	Ensure satisfaction with follow-up communication.

“A prompting technique that encourages the model to reason step-by-step, mimicking human logical thought processes.”

Chain of Thought

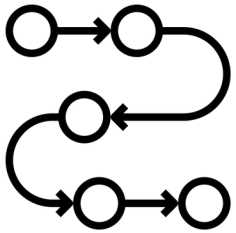
4 STEP FRAMEWORK

1. Identify the main goal or objective of the prompt.
2. Break down the main goal into sub-tasks, that is, more specific actions or tasks.
3. Create a prompt for each specific action or task.
4. Ensure that each prompt is clear, concise, and unambiguous.

BALANCING CoT USAGE

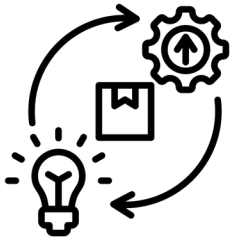
1. **Is the task complex or multi-step?**
   Use CoT if yes.
2. **Does the problem require reasoning or logical steps or has dependent systems?**
   Use CoT if yes.
3. **Would step-by-step reasoning improve transparency Of output?**
   Use CoT if yes.
4. **Is speed or simplicity more critical than detailed reasoning?**
   Avoid CoT if yes.

Advanced Chaining Techniques



Sequential Prompting

Conditional Prompting



Iterative Prompting

Sequential chaining involves linking prompts in a straightforward, linear sequence. Each prompt depends on the output of the previous one, creating a step-by-step flow of information and tasks.

Sequential Prompting

Sequential prompting is best suited for structured, repeatable, and comprehensive workflows but is less effective in dynamic, simple, or open-ended scenarios. It ensures thoroughness but lacks flexibility.

Conditional chaining introduces branching in the prompt chain, allowing flexible workflows where the next step adapts based on the LLM's output.

Conditional prompting is ideal for dynamic, context-sensitive workflows that benefit from branching and flexibility. However, it is less suitable for rigid, simple, or highly standardized tasks where fixed processes are more effective.

Iterative prompting is a technique where you refine and improve prompts over multiple iterations to guide the LLM toward a better and more accurate response.

Iterative Prompting

Iterative prompting is best suited for refining vague outputs, exploring complex problems, and improving creative tasks. It is less effective for simple, time-sensitive, or highly structured workflows where straightforward approaches suffice.

A reusable, structured framework for crafting effective prompts when interacting with large language models (LLMs).

Prompt Template

01

Identify Frequent Issues:

- List recurring customer problems or support scenarios.

02

Determine Key Information:

- Define what the LLM needs to address the issue effectively.

03

Structure the Template:

- Write a clear, reusable prompt format with placeholders for dynamic inputs.

04

Test the Template:

- Run the template on real-world examples and refine for clarity and accuracy.

Few important points :

- Please keep your Copilot open and accessible
- Be ready to take part in polls & quizzes
- Be ready to work on hands-on exercises

Live Demo

BREAK (7 MINS)

MORE PROBING TECHNIQUES

1. Tree of Thoughts
2. Multimodal CoT (Vision Model)
3. Reasoning + Acting
4. RAG

Advanced Probing Techniques

TREE of THOUGHTS

Tree of Thought prompting involves branching and exploring multiple reasoning paths simultaneously, where each thought represents an intermediate step toward solving a problem. It focuses on generating and evaluating multiple possibilities, at each step iteratively refining the path to the solution.

Tree of Thoughts

Test Customer Query

"Debugging an authentication issue for a Microsoft Azure app"

.

Tree of Thought(ToT) Prompting

Step 1: Generate thoughts

Prompt: "List all possible reasons why a Microsoft Azure app might fail authentication when using MSAL for device code flow. Include issues related to configuration, permissions, and network setup."

Copilot Response:

1. Incorrect redirect URI configuration.
2. Missing API permissions in Azure portal.
3. Expired or invalid client secret.
4. Firewall blocking access to Microsoft endpoints.
5. Tenant mismatch in authority URL.

Tree of Thought (ToT) Prompting

Step 2: Explore a specific branch

Prompt: "For issue 1 (Incorrect redirect URI configuration), describe how to verify and correct the redirect URI in the Azure portal and the application code."

Copilot Response:

1. Navigate to the Azure portal and open the app registration.
2. Under Authentication, ensure the redirect URI matches the one in the app.
3. Update the app configuration file (authConfig.json) with the correct redirect URI.

Tree of Thought (ToT) Prompting

Step 3: Evaluate progress

Prompt: "Given the steps described, how likely is this solution to resolve the authentication issue? If uncertain, suggest further diagnostics."

Copilot Response:
This solution is highly likely to resolve the authentication issue if the root cause is a mismatch in the redirect URI. If the issue persists:

1. Check if there are multiple redirect URIs configured for different environments or deployment stages (e.g., testing, production).
2. Examine any HTTP or browser redirects for errors.
3. Review logs for any mismatch warnings related to redirect URI.

Tree of Thought (ToT) Prompting

Step 4: Refine and Iterate

Prompt: "If redirect URI configuration doesn’t resolve the issue, generate diagnostic steps for investigating API permission problems."

Copilot Response:

1. Check API Permissions in Azure Portal:
    - Navigate to Azure Active Directory > App registrations > Select the app.
    - Under API permissions, ensure that required permissions (e.g., User.Read, Profile.Read) are added.
    - If missing, add necessary permissions and ensure they align with the app's requirements.
2. Admin Consent:

Tree of Thought (ToT) Prompting

Step 5: Explore another branch

Prompt: "For issue 4 (Firewall blocking access to Microsoft endpoints), outline how to identify and resolve firewall issues affecting authentication."

Copilot Response:

- **Identify Blocked Endpoints**:
  - Review firewall logs and check if requests to Microsoft authentication servers (e.g., `login.microsoftonline.com`) are being blocked.
  - Use network diagnostic tools (e.g., `ping`, `tracert`, `nslookup`) to check connectivity to Microsoft endpoints.
- **Allow Necessary Endpoints**:
  - Configure the firewall to allow access to required Microsoft endpoints.
  - Verify that ports commonly used for authentication (e.g., 443 for HTTPS) are open and not being restricted by the firewall.

Tree of Thought (ToT) Prompting

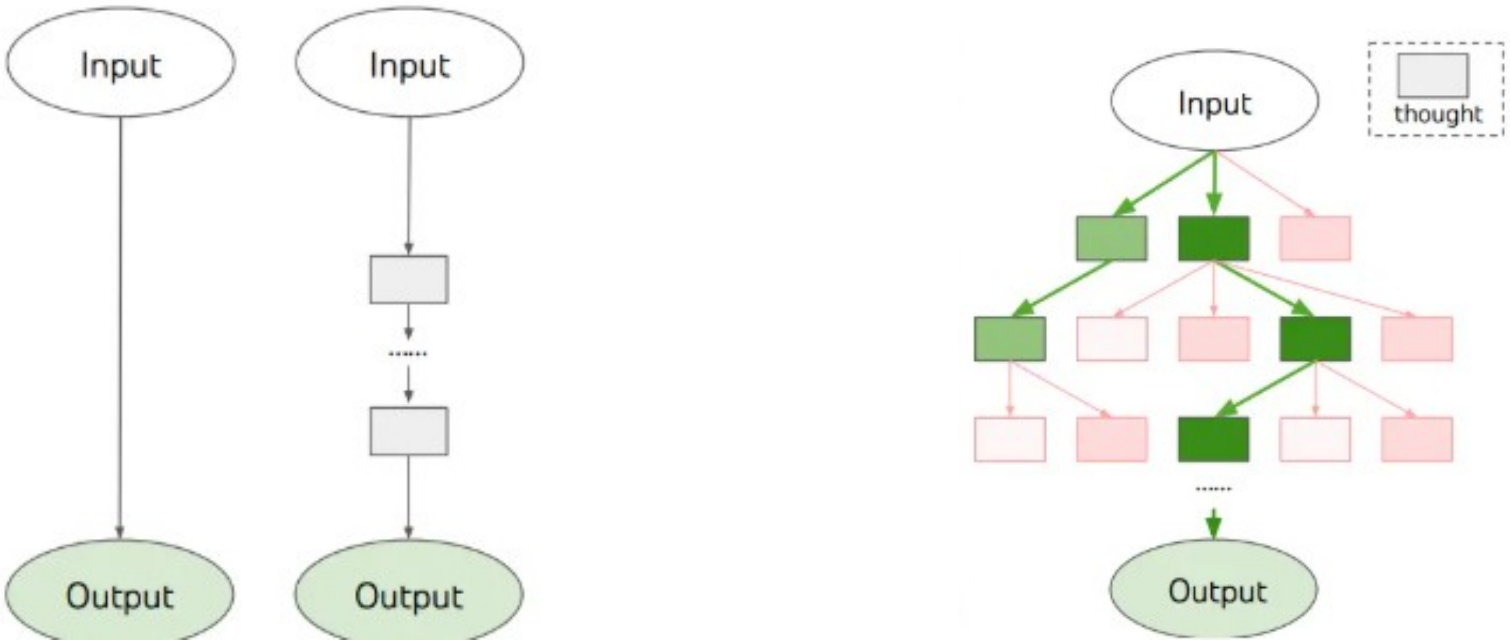
Step 5: Evaluate overall progress

Prompt: "How likely are the above steps to resolve the authentication issue? What further actions can be considered?"

Copilot Response:
These steps are likely to address the issue if the problem is related to firewall configuration. However, if the issue persists:

1. Use Azure AD sign-in logs to capture detailed error messages and authentication attempts.
2. Test the authentication flow in different environments to isolate any specific network or configuration-related issues.

Tree of Thought (ToT) Prompting



Tree of Thought(ToT) Prompting

“Imagine three different experts are answering this question. All experts will write down 1 step of their thinking, then share it with the group. Then all experts will go on to the next step, etc. If any expert realises they're wrong at any point then they leave. The question is...

Debugging an authentication issue for a Microsoft Azure app”

Poplar Prompt Structure

1. **Clearly Define the Problem and Goals (Root of the Tree) :** A well-defined problem ensures you start with clarity and purpose, avoiding confusion or irrelevant branches.
   1. State the problem clearly, including any observations, constraints, and desired outcomes.
   2. Break down the issue into smaller, actionable components that form the foundation of your "tree."

2. **Systematically Explore and Prioritize Branches :** Organized exploration ensures no critical paths are overlooked and reduces wasted effort on unlikely causes.
   1. Start with the simplest, most likely, or highest-impact branches.
   2. Use observations and diagnostic results to dynamically prioritize branches as you progress.

3. **Reflect and Iterate at Each Step:** Reflection helps refine the approach based on outcomes, avoiding dead ends and enabling informed next steps.
   1. After each action, evaluate the results to decide whether to continue down the branch or try another.
   2. Adapt reasoning dynamically based on the feedback.

BEST PRACTICES

In the Tree of Thought (ToT) framework, which practice ensures the problem-solving process remains structured and avoids missing critical paths?

A. Clearly defining the problem and goals.

B. Exploring and prioritizing branches systematically.

C. Reflecting on outcomes at each step.

D. Documenting actions and results.

P1

60 Seconds

Multimodal Prompting

Multimodal CoT prompting combines both text-based reasoning and visual cues (such as images, diagrams, or screenshots) in the problem-solving process.

Multi Modal

Understand Through Demo

What is the primary benefit of using multimodal prompting techniques in technical support?

A. Integrating data from multiple sources like text, images, and logs for holistic problem-solving.

B. Improving the model's ability to generate creative and non-technical responses.

C. Reducing the complexity of troubleshooting by focusing on a single input type.

D. Limiting the scope of analysis to avoid overloading the system.

P2

60 Seconds

ReAct

The ReAct (Reasoning + Acting) prompting structure is designed to combine logical reasoning with actions to solve complex problems or perform tasks iteratively.

ReAct

1. **Task Definition** :Clearly define the overall goal or task for the model. Specify what the model is expected to achieve.

2. **Input:** Observation Provide the relevant context, observations, or data (e.g., logs, error messages, user reports). This serves as the initial input for reasoning.

3. **Reasoning :** Ask the model to analyze the input, draw logical conclusions, and break the problem into smaller components. Encourage step-by-step reasoning for clarity and accuracy.

4. **Action :** Guide the model to take a specific action or suggest the next step in resolving the problem.

5. **Reflection:** After the action, ask the model to reassess the situation based on the results or outcomes and decide on the next course of action. This is where iterative refinement happens.

ReAct PROMPT STRUCTURE

Ticket Details
Customer Name: John Doe
Operating System: Windows 11
Issue Reported: Windows Update fails with error code 0x80070057. The customer has attempted restarting the system, but the issue persists.

Customer Ticket

Task: Diagnose and resolve a Windows Update issue using step-by-step reasoning and actions.

Observation: The customer reports error code `0x80070057` during a Windows Update on Windows 11. They also mentioned restarting the system with no success.

Reasoning:
- What does the error code indicate?
- What could be the root cause of the issue?
- Based on the observations, what steps should we try first?

Action: Suggest specific troubleshooting steps to resolve the issue, starting with common fixes.

Reflection:
- If the suggested action resolves the issue, explain why it worked.
- If not, refine the approach and suggest the next action.

ReAct Prompt

QUICK LIVE DEMO

1. **Incorporate Knowledge Bases:** Leverage Microsoft’s extensive documentation (e.g., Windows Troubleshooting, Azure Knowledge Base) as part of the reasoning process.

2. **Set Clear Observations:** Ensure the problem description is detailed and contextualized.

3. **Define Actions Precisely:** Each action should be specific and immediately applicable (e.g., "Run Get-WindowsUpdateLog to gather update logs.").

4. **Iterate Based on Results:** Use reflections to determine whether the previous action was successful or if additional steps are needed.

BEST PRACTICES

In the ReAct framework, which step ensures that the solution is iteratively refined and improved based on outcomes?

A. Observation

B. Reasoning

C. Action

D. Reflection

P3

60 Seconds

RAG

RAG combines retrieval from external knowledge sources with the generative capabilities of large language models (LLMs).

1. **Instruction Layer :** This defines the task or the specific goal you want the model to achieve. Be clear and concise.

2. **Context Layer :** This includes the retrieved information (e.g., knowledge base documents, articles, logs). Use delimiters like ###, Context:, or similar markers to denote retrieved information.

3. **Query or Input Layer :** This includes the user's query or the specific problem to be solved. Clearly frame the question or request for the model.

4. **Response Instructions (Optional) :** Add specific instructions about the expected format of the output (e.g., bulleted steps, concise paragraphs, or detailed explanations).

RAG PROMPT STRUCTURE

Instruction:
Using the provided context, answer the following question and provide a clear, step-by-step solution.

Context:
1. Error code 500 is often caused by incorrect environment variable configurations.
2. Solution: Ensure that the `.env` file contains valid database credentials and matches the expected schema.
3. Documentation Reference: Troubleshooting Guide - https://example.com/troubleshooting-500

Query:
Why am I encountering error code 500 during deployment, and how can I fix it?

Response Instructions:
Provide the solution in bullet points and include references to the context when applicable.

EXAMPLE

1. **Be explicit:** Specify how the model should use the retrieved context. "Base your answer only on the retrieved information." Separate context clearly: Use delimiters (###, "Context:") to make the retrieved information distinct.

2. **Provide fallback instructions:** If the context is insufficient, instruct the model on what to do. "If the context does not fully answer the question, explain why and suggest next steps."

3. **Limit context length:** Include only the most relevant portions of the retrieved data to avoid overwhelming the model (consider token limits). Don't give too many external links.

4. **Guide the response format:** Specify the structure of the output, such as bulleted lists, numbered steps, or detailed paragraphs.

BEST PRACTICES

QUICK LIVE DEMO

Which step is NOT typically part of the Retrieval-Augmented Generation (RAG) workflow?

A. Retrieve relevant documents or data from an indexed knowledge base.

B. Generate a response by combining retrieved data with reasoning.

C. Execute direct changes in a user’s system without human validation.

D. Refine results iteratively by tapping knowledge base to ensure accuracy.

60 Seconds

LIVE DEMO

Break (7 Mins)

Hands-On (15 Mins)

Q&A